

PROJECT REPORT

C Lexical Analyzer (Scanner)

Developed in: C Programming Language

Architecture: Command-Line Interface (CLI)

Target Application: Compiler Design (Phase 1)

1. INTRODUCTION

Lexical Analysis is the fundamental first phase of any compiler architecture. The C Lexical Analyzer is a custom-built, command-line scanning utility that reads raw C source code, strips away irrelevant formatting (such as whitespace and comments), and groups the remaining characters into a structured stream of meaningful units called Tokens. By building this analyzer manually from scratch rather than relying on automated generators like Lex or Flex, the project demonstrates a profound understanding of deterministic finite automata (DFA), string manipulation, state tracking, and low-level C file I/O operations.

2. SYSTEM ARCHITECTURE & MODULES

The system is strictly modularized to separate input validation from the core lexical processing engine.

2.1. Execution Controller (main.c)

This module serves as the command-line interface. It enforces execution constraints, validating that an argument is provided and that the target file ends with the `.c` extension. Upon successful validation, it securely opens the file descriptor and passes the stream to the lexical engine.

2.2. Lexical Engine (lexer.c & lexer.h)

The core engine manages line-by-line file streaming using `fgets`. It iterates character-by-character, identifying delimiters (spaces, commas, semicolons) to isolate strings. Isolated strings are then forwarded to the `categorizeToken()` function, which leverages an enumerated `TokenType` standard to classify the input into one of eight distinct types: Keywords, Operators, Special Characters, Constants, Identifiers, Preprocessor Directives, Comments, or String Literals.

3. TOKENIZATION LOGIC & STATE MANAGEMENT

The engine enforces strict C-language standard rules to validate and categorize lexical patterns.

Identifier Validation

To differentiate variables from syntax errors or constants, the `isIdentifier()` function explicitly enforces standard compiler nomenclature rules. It validates that the token begins exclusively with an alphabetical character or underscore, followed strictly by alphanumeric values.

```
int isIdentifier(const char* str) {
    // Cannot start with a number or special symbol
    if(!isalpha(str[0]) && str[0] != '_') return 0;

    // Remaining characters must be alphanumeric
    for(int i = 1; str[i] != '\0'; i++) {
        if(!isalnum(str[i]) && str[i] != '_') return 0;
    }
    return 1;
}
```

4. KEY FEATURES & OPERATIONS

Feature	Implementation Details
String Literal Tracking	Utilizes a state flag (<code>s1</code>) to suspend standard delimiter parsing when an opening quote (<code>"</code>) is encountered, safely aggregating space-separated words into a single String Token until the closing quote is reached.
Comment Stripping	Detects sequential slashes (<code>//</code>) or block markers (<code>/*</code>) to immediately halt code processing, tagging the entity as a COMMENT and preventing syntax misinterpretation.
Directive Handling	Monitors the leading index of the line buffer for the hash (<code>#</code>) symbol to classify includes and macros specifically as Preprocessor Directives.

5. LIMITATIONS AND FUTURE ENHANCEMENTS

To upgrade this lexical scanner to a production-grade compiler component, the following architectural optimizations are targeted:

- **Maximal Munch Rule & Lookaheads:** The current character-by-character state machine may incorrectly parse multi-character operators (e.g., `==`, `<=`, `++`) as independent single operators. Implementing a 1-character lookahead buffer will resolve these ambiguities using the Maximal Munch principle.
- **Dynamic Buffer Allocation:** Tokens are currently aggregated into a static `char word[50]` buffer. If a source file contains an identifier exceeding 49 characters, it introduces a buffer overflow vulnerability. Transitioning to a dynamically resizable heap array via `realloc` is necessary for memory safety.
- **Multi-Line Comment States:** Because the `fgets` loop resets operational scope per line, multi-line comments (`/* ... */`) lose their state context on the subsequent line. Introducing a global `in_comment` state flag will harden comment stripping across newline boundaries.
- **Hash-Table Keyword Lookup:** The current `isKeyword()` implementation utilizes an $O(N)$ linear array search. Given the high frequency of keyword checks, implementing a Perfect Hash Function or Trie structure will reduce lookup time to $O(1)$.

6. CONCLUSION

The C Lexical Analyzer project successfully distills a highly complex phase of compiler design into an actionable, manual implementation. By meticulously handling raw string manipulation, standardizing memory states, and enforcing rigid C syntax constraints, this project stands as a testament to deep comprehension of systems-level programming and text processing algorithms.